

University of Pittsburgh

Introduction to Operating Systems
CS/COE 1550

Process Synchronization - 1

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

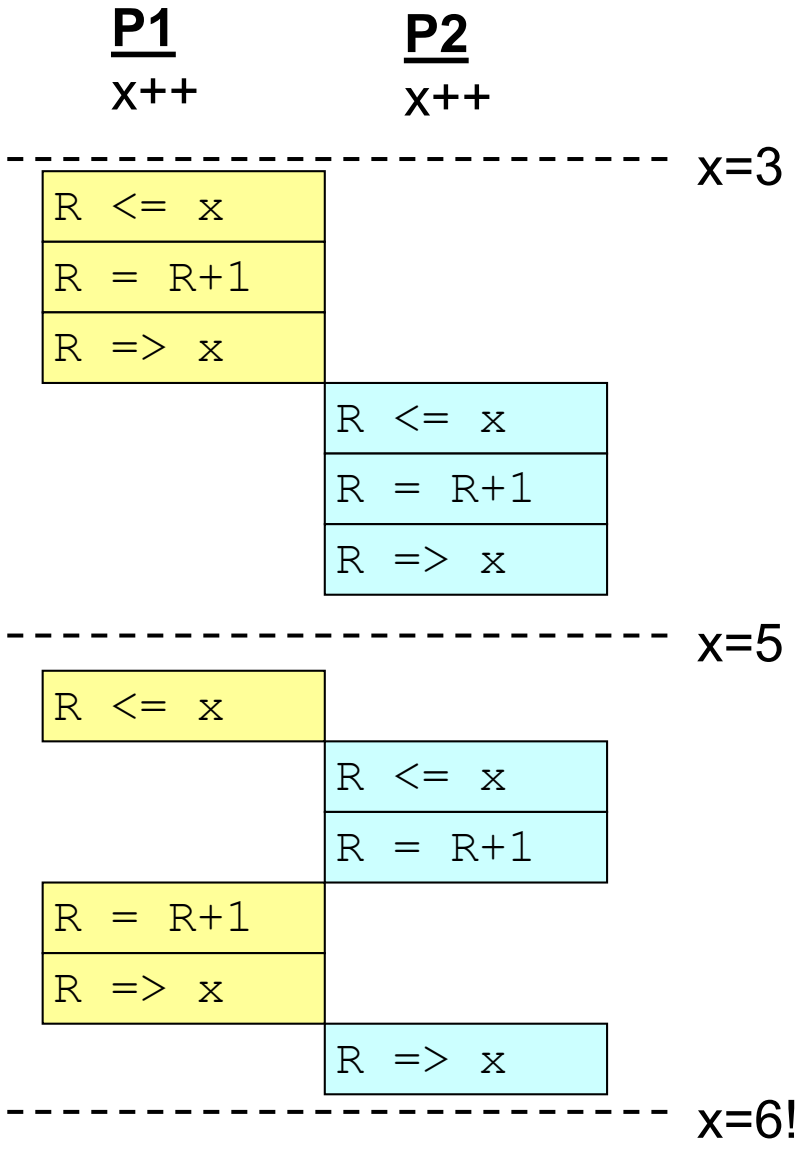
- Homework 1 is due next Monday at 11:59 pm
- Recitations started last week
  - Please watch the videos
- Lab 1 is due on 2/4 at 11:59 pm
  - Distributed using GitHub Classroom
  - Explained in this week's recitations
- Steps of a syscall posted on Canvas
- TA Office hours available on the syllabus page
- Muddiest points will be anonymous and not seen to students

Muddiest Points

- storage/access of the mode bits
- syscalls
  - idt/isr vs syscall table
  - Difference between f1~f7(what to call them?) and f8~f10(syscall functions)
  - the beginning of the syscall process
  - syscall steps
  - eax register relevance to interrupts/syscall
  - How user programs need to be changed for new syscalls
  - How high level code interacts with syscalls
- xv6 walkthrough
  - Array of functions syntax in C
- monolithic vs micro

Problem: race conditions

- R is a CPU register
- X is a variable stored in memory

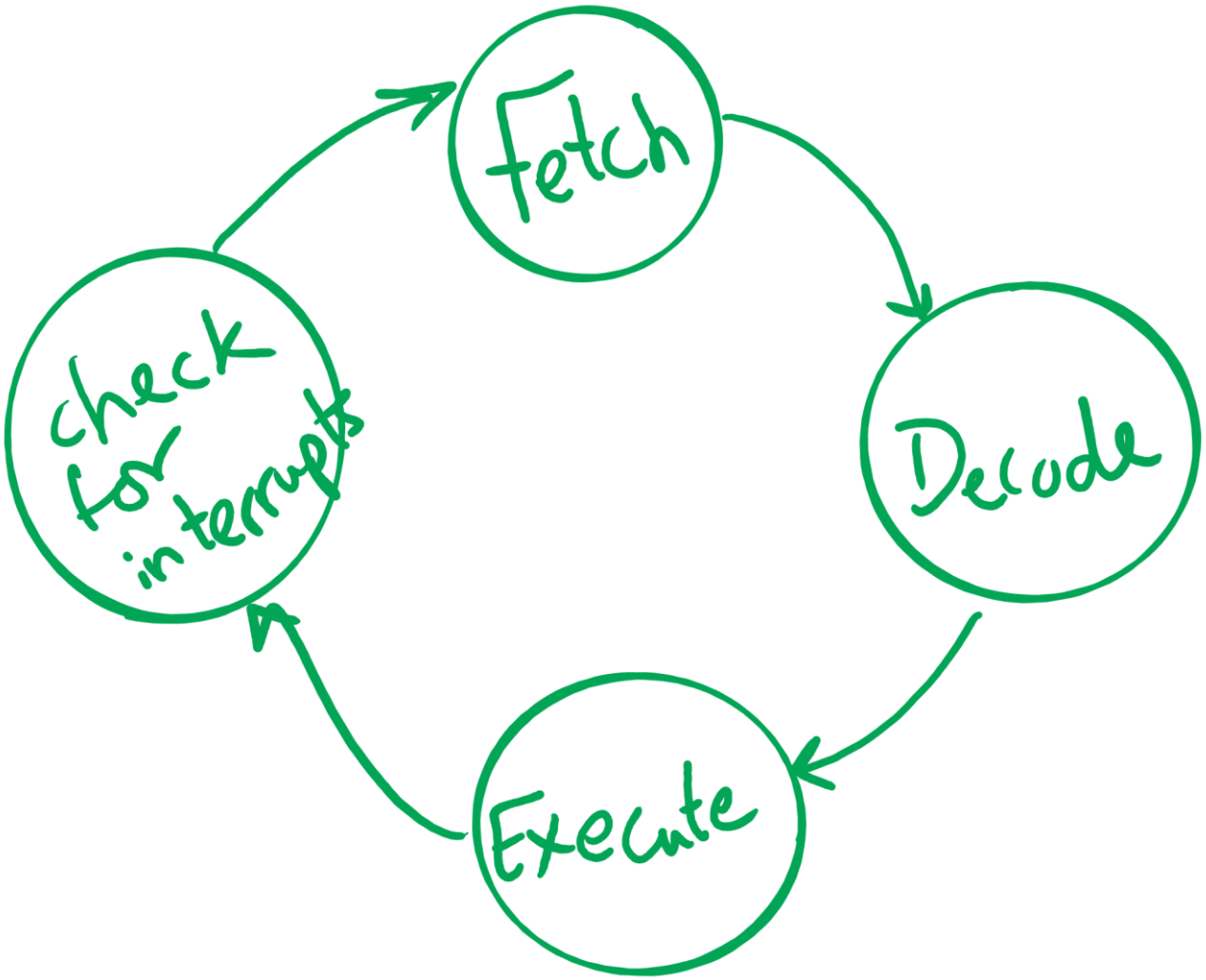


Race conditions

- Cooperating processes share storage (memory)
- Both may read and write the shared memory
- Problem: can't guarantee that read followed by write is **atomic**
  - Atomic means uninterruptible
  - Ordering matters!
- This can result in erroneous results!
- We need to eliminate race conditions…

Atomic operations

- If done in one instruction, then not interruptible



Context Switching

How did the CPU switch from P1 to P2 then to P1 then to P2 again …?

Process Control Block

``Active entities are data structures when viewed from a lower level.''

Raphael Finkel, University of Kentucky

Process Control Block (PCB)

Information associated with each process

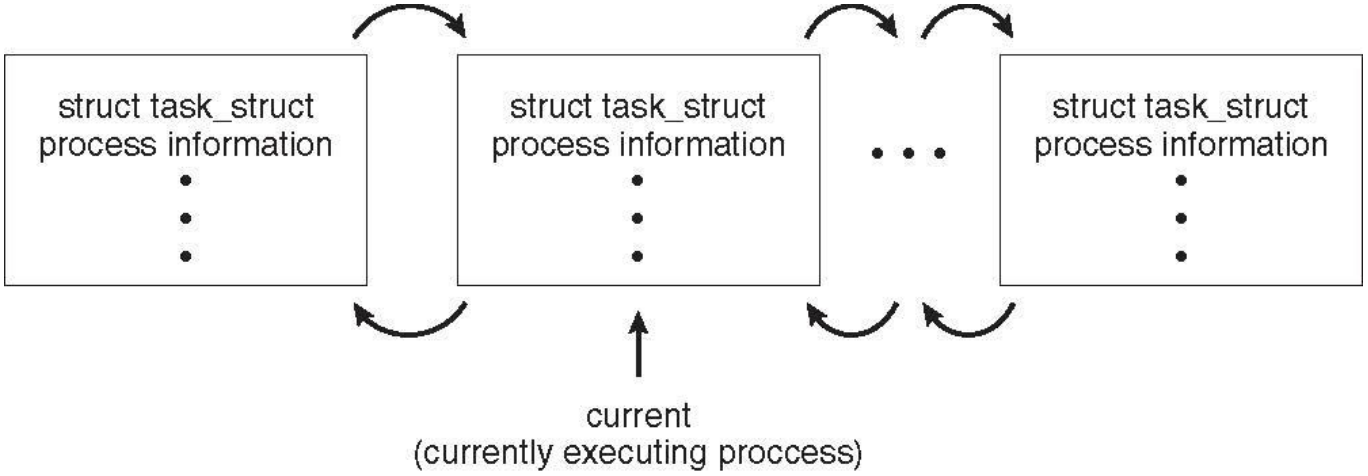
(also called **task control block**)

- Process state – running, waiting, etc
- Program counter – location of instruction to execute next
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

Process Representation in Linux

Represented by the C structure `task_struct`

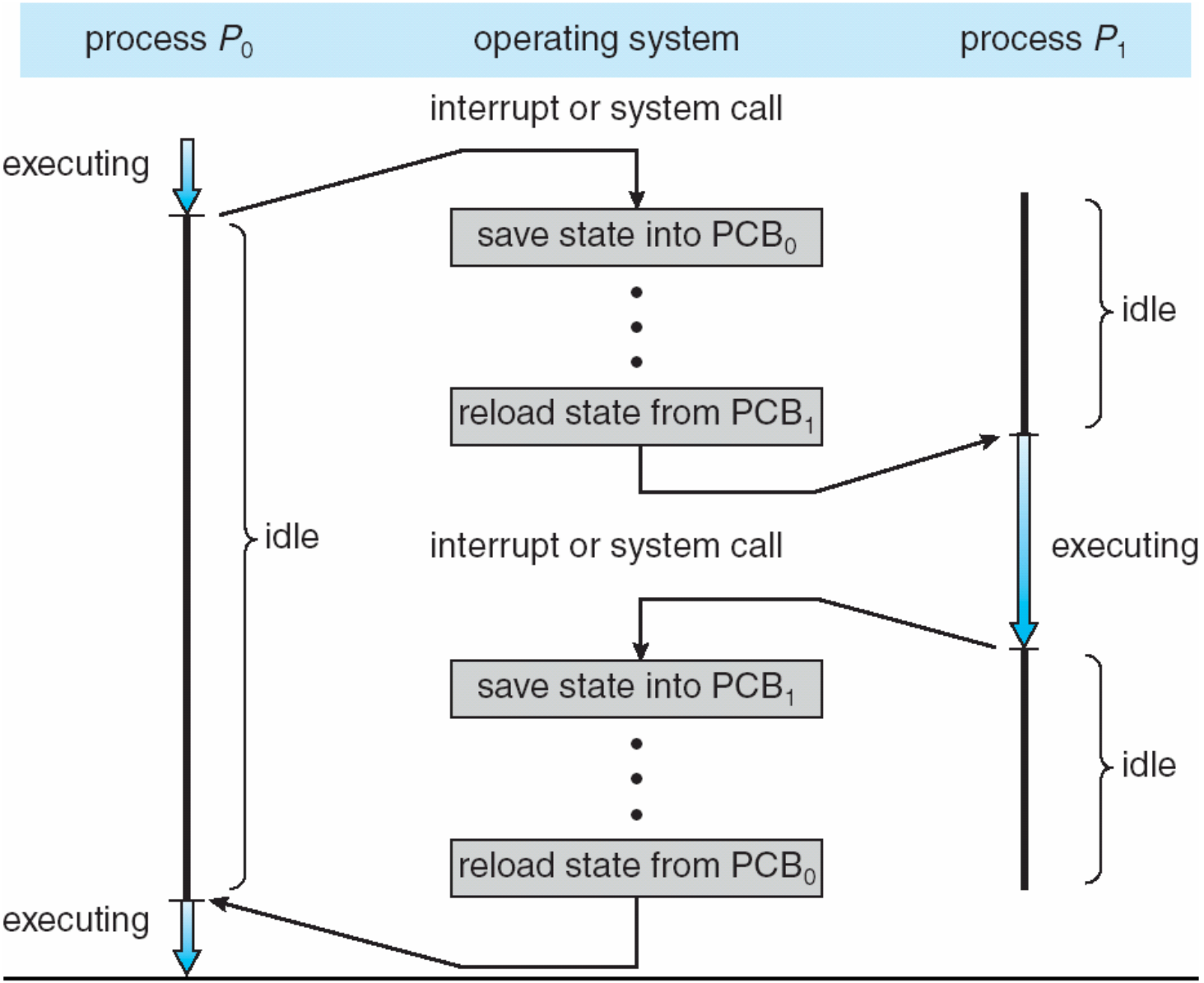
```
pid_t pid; /* process identifier */
long state; /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this process */
```



Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once

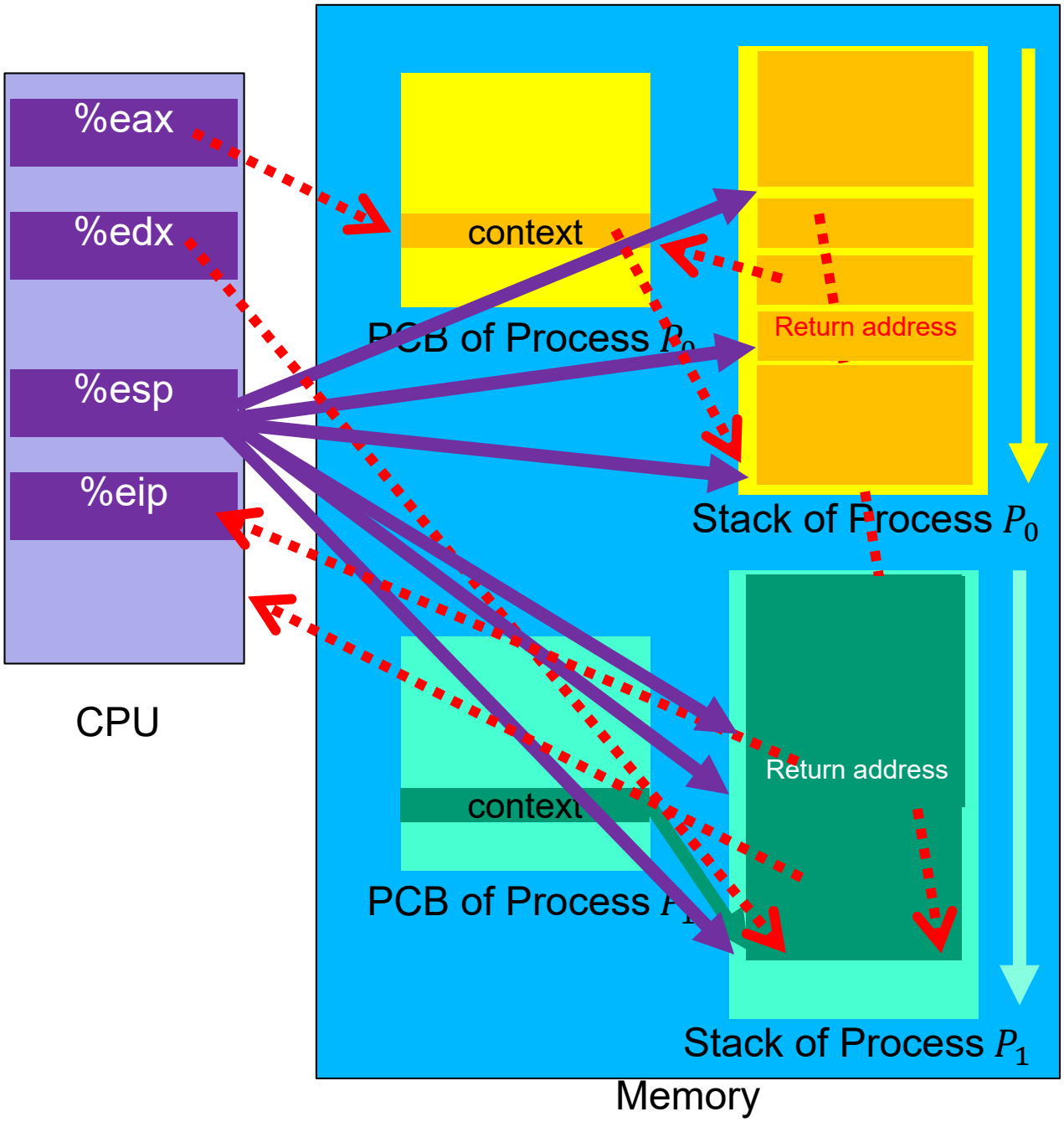
Context Switching



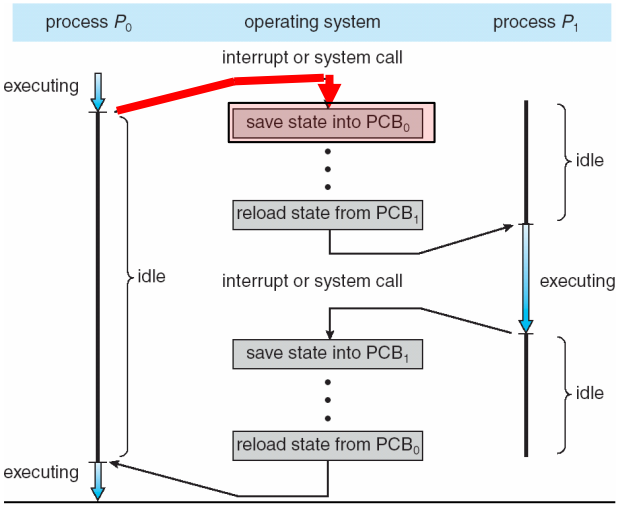
Xv6 Code Walkthrough

- PCB and process table
- Context switching
- Calling of the swtch routine

Context Switching in Xv6

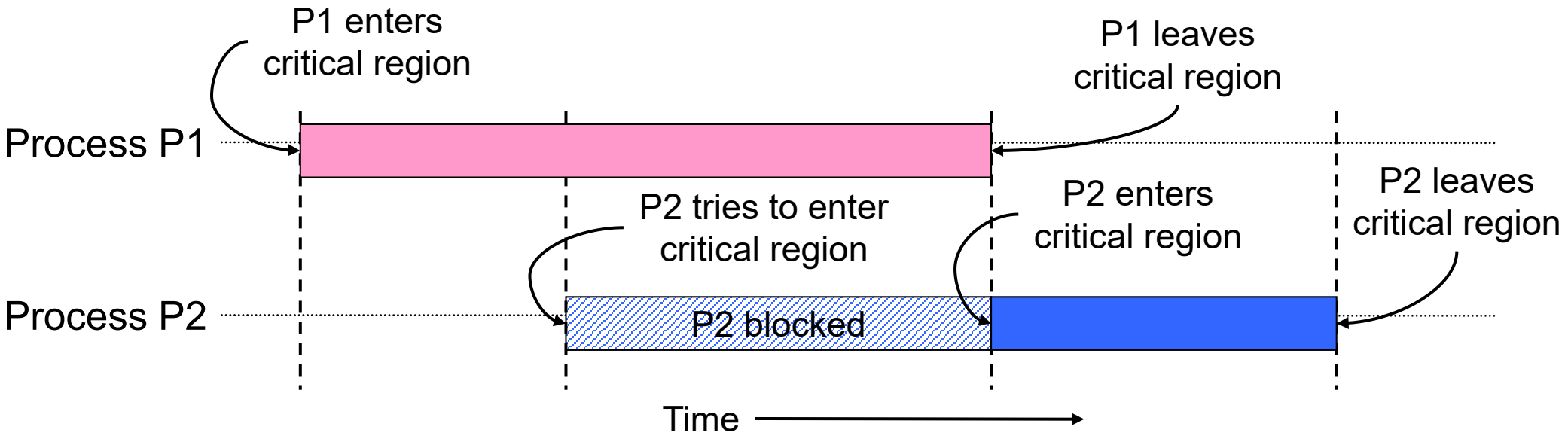



```
1   .globl swtch
2   swtch:
3     movl 4(%esp), %eax
4     movl 8(%esp), %edx
5
6     # Save old callee-saved registers
7     pushl %ebp
8     pushl %ebx
9     pushl %esi
10    pushl %edi
11
12    # Switch stacks
13    movl %esp, (%eax)
14    movl %edx, %esp
15
16    # Load new callee-saved registers
17    popl %edi
18    popl %esi
19    popl %ebx
20    popl %ebp
21    ret
```



Critical regions

- Back to the race conditions problem
- Use critical regions to provide *mutual exclusion* and help fix race conditions
- Let's put the statement x++ in a critical region



How to implement critical regions?

- Turn-based solutions
- Spinlocks
- Semaphores
- Monitors

Using Spinlocks

```
Spinlock lock;
```

Code for process P_i

```
While(1){
   Lock(lock)
   // critical section
   Unlock(lock);
   // remainder of code
   }
}
```

Spinlock implementation (1/2)

- Solution: use hardware
- Several hardware methods
  - Test & set: test a variable and set it in one instruction
  - Atomic swap: switch register & memory in one instruction
  - Turn off interrupts: process won't be switched out unless it asks to be suspended
- The first two methods can be implemented in user land
  - Why can't we implement the third method in user land?